

Business Harness Bench

Research Premise and Benchmark Specification

Release snapshot | 17 September 2026

Abstract

The relevant output of a business agent is not its explanation of the work, but the work it leaves behind. A reconciliation must account for the right transactions, an import must preserve the destination's contract, and an application must enforce its permissions when someone actually uses it. Business Harness Bench evaluates this handoff through two complementary tracks: 187 desk tasks that transform folders of business files into checkable deliverables, and 20 application-building tasks that require a working application followed by three change requests. Desk tasks combine explicit business rules, heterogeneous inputs, deterministic checks, and reference solutions. Build tasks combine seed-data requirements with an enterprise acceptance baseline covering access control, sharing, live data, auditability, and persistence. The unit of comparison is the configured agent system: harness, model, tools, and execution substrate. Repeated attempts distinguish occasional success from repeatable completion; time, recorded usage, and estimated model cost describe the resources required. This release includes the four complete desk arms of the latest full benchmark campaign, totaling 2,244 attempts, and does not present incomplete application evaluations as a build leaderboard. The benchmark is a controlled test of specified business-work contracts, not a claim to represent all business activity or to establish unattended production readiness.

1. The problem: completion is a contract, not a conversation

A small business does not delegate a reconciliation to obtain a convincing description of reconciliation. It delegates the work because a payment decision, a report, or the next import depends on the result. The handoff is therefore the right place to evaluate an agent. Can the owner open the files, trust the specified calculations, identify what remains unresolved, and continue the business process without repairing the deliverable?

The distinction matters because business correctness is often conjunctive. A workbook can contain the right total while using broken formulas. An import can be well-formed while silently dropping customers. A payment can match an amount while belonging to a different invoice. A summary can sound polished while recommending an option contradicted by its sources. In an application, a hidden button is not an authorization boundary, and a successful demonstration is not evidence that records survive a restart. These are not cosmetic shortcomings around an otherwise correct answer. They break the contract under which the work was delegated.

Business Harness Bench makes that contract explicit and executable where possible. The central question is: **under a stated operating environment and resource budget, how reliably does a configured agent system deliver business work that satisfies the owner's requirements?** Success is attached to the resulting artifacts and observable behavior, not to confidence, verbosity, a self-reported completion message, or the number of actions performed.

2. Two handoffs, one evaluation principle

The desk track starts with a folder and a short request. The agent receives business-shaped exports, spreadsheets, documents, and contextual instructions, then leaves the requested deliverables. Tasks test not only transformation mechanics but the application of supplied rules: which record is authoritative, how an exception should be treated, what must remain unchanged, and when the evidence is insufficient to support a conclusion. Inputs are generated fixtures, not a claim that the suite consists of privately collected customer engagements. The tasks use ordinary formats so the tested capability is producing the deliverable, rather than access to a proprietary input representation.

The build track starts with a business requirement and seed data. The agent must hand over an application that a tester can reach and operate, then accommodate three changes without discarding earlier requirements. The initial build tests delivery; the subsequent turns test continuity. A system that can produce a plausible first screen but cannot preserve access restrictions or existing records across a change has not completed the same business job as a system that can. Automated probes support this evaluation, but they do not substitute for the unautomated acceptance checklist.

The tracks are reported separately. They share a completion principle, not a common denominator: passing a file check is not interchangeable with enforcing an application's role boundary. Combining them into one headline score would obscure what was actually tested.

3. Evaluate the system that does the work

A model does not act alone. Its harness determines how it receives instructions, reads files, invokes tools, manages context, recovers from errors, and delivers artifacts. Available skills, document utilities, model routing, and an application platform can change both the feasible solution and its cost. The benchmark therefore identifies each experimental cell by its harness, model configuration, tools, and substrate.

An end-to-end comparison answers a buyer's question: which configured system completed this workload under these conditions? It does not, by itself, isolate the causal contribution of the harness or establish that one underlying model is better. Those questions require matched controls, including the same model and substrate where possible. Likewise, access to a business-app platform is a declared experimental condition, not an invisible advantage. A useful result describes the system that ran, including material configuration gaps, rather than attributing every difference to one component.

4. Reliability and resources belong beside accuracy

A single successful attempt establishes possibility, not repeatability. The desk protocol uses three attempts per task and reports both attempt-level pass rate and the fraction of tasks that pass all three attempts. The former describes performance across the executed matrix; the latter identifies work completed consistently within those repetitions. Neither estimates long-term reliability with high precision from only three observations per task.

Every included full arm retains failures as well as successes. Execution errors, timeouts, missing usage, and grader errors remain visible instead of being silently removed from the denominator. Artifact checks and normal process completion are distinct observations: an agent may leave a passing file before timing out, or exit normally without delivering correct work. Reporting both prevents a convenient process status from standing in for the business outcome.

Time and estimated cost further qualify the result. A correct deliverable that requires substantially more time or expenditure represents a different operating point. Recorded token counts and the bundled price table support a transparent estimate; they are not an invoice and do not account for every infrastructure, subscription, or human-review cost. The benchmark's purpose is to expose these tradeoffs, not to collapse them into an unexplained composite score.

5. The evaluator must also earn trust

A deterministic grader is repeatable, but not automatically correct. A check can reject a valid alternative, accept a superficial match, or rely on a stale spreadsheet cache. The task package therefore includes explicit checks and reference solutions, and its validation tools test regeneration and basic positive and negative cases. Native spreadsheet recalculation is important when the contract requires live formulas. These mechanisms make the scoring inspectable; they do not establish exhaustive semantic coverage.

The scope of the claim must stay aligned with the evidence. Mechanical authoring checks are not independent practitioner review. A public generated suite is not an unseen holdout. Benchmark-specific development creates exposure that must be disclosed rather than relabeled as generalization. Recorded campaign scores without a per-attempt scorer digest are a historical scoring snapshot, not proof that every artifact was evaluated by an identical immutable scorer. Missing build acceptance work is missing evidence, not a zero that can be filled by an optimistic narrative.

This is the benchmark's intended contribution: a runnable, inspectable way to evaluate whether agents satisfy concrete business-work contracts, with enough separation between task validity, execution, grading, and reporting to make disagreements diagnosable. The release is not an account of one product's tuning history, and its premise does not depend on which system ranks first.

6. Positioning and boundaries

Business Harness Bench sits alongside existing approaches to evaluating work. WorkArena evaluates agents performing knowledge-work tasks in enterprise software. SpreadsheetBench evaluates spreadsheet manipulation grounded in real-world scenarios. TheAgentCompany evaluates professional tasks in a simulated workplace. These works establish relevant neighboring settings; this paper does not claim that business work or artifact-based evaluation is new. The particular emphasis here is the combination of portable file-delivery contracts and application handoff with subsequent changes, reported at the configured-system level.

The present suite contains authored business-shaped tasks, not a representative sample of the economy. Task families may share generators and failure modes; three attempts per task are not independent samples of business demand. Document checks do not establish editorial quality in every respect, and file tasks do not exercise live SaaS state, organizational coordination, or every security property. Platform-backed builds require appropriately scoped external access. Results should guide investigation and comparison within this defined workload, not be presented as a guarantee of safe autonomous deployment.

References

1. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks?
<https://arxiv.org/abs/2403.07718>
2. SpreadsheetBench: Towards Challenging Real World Spreadsheet Manipulation.
<https://arxiv.org/abs/2406.14991>
3. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks.
<https://arxiv.org/abs/2412.14161>

7. Task inventory and contracts

The release contains 187 desk tasks and 20 build tasks. Every task includes a generator and evaluator-side materials. Only workspace inputs and the owner's request are provided to the evaluated agent; reference answers, check definitions, traps, and task author notes are not mounted into the agent container.

Desk category	Tasks	Typical deliverable
Spreadsheet	41	Reconciled, calculated, or reshaped workbook
Bookkeeping	36	Reconciliation, schedule, transaction classifications
Reports	30	Data summary and source-grounded memo
Reformatting	26	Destination-compatible import file
Extraction	26	Structured records extracted from documents
Drafting	19	Business text preserving required facts and rules
Tooling	9	Small file-based tool or static page
Total	187	Three repetitions per complete desk arm

A desk task directory contains `task.yaml`, `gen.py`, `workspace/`, `reference/`, `reference_solution/`, and, when required, `check.py`. The YAML declares the ask, category, time budget, and checks. Files in `reference_solution/` provide a positive control, not an agent solution to ship in the run workspace. A generator creates reproducible task fixtures; a regenerated seed is a related variant, not evidence of an independently sourced held-out task distribution.

A build task directory contains `task.yaml`, `gen.py`, `seed/`, `reference/counts.json`, `checklist.md`, and three change-request files. The application inventory covers CRM, inventory, orders, field service, leave, purchase approvals, time and invoicing, memberships, events, assets, helpdesk, vendors, expenses, quotes, donors, appointments, property maintenance, recruiting, work orders, and point of sale.

The build acceptance baseline covers delivery, invitations, role management, server-side access control, scoped live dashboards, exact lists/exports, audit metadata, validation, and persistence. App-specific requirements add domain behavior. The supplied checklists, including the field-service task's distinct baseline placement, are authoritative; no uniform item count is inferred merely from the number of tasks.

8. Execution protocol

8.1 Agent boundary

The desk runner copies only the task workspace into an opaque temporary directory, writes the request separately, and invokes a harness adapter with three arguments: workspace, prompt file, and output directory. In Docker mode the agent sees `/run/ws` and the per-run home; the benchmark task tree and reference answers stay on the host. Read-only adapters are mounted separately. The container runs as the host user with CPU, memory, and process-count limits. Network access remains available for model requests and permitted tooling; this is not an offline benchmark or a complete defense against discovering published tasks online.

Local-process mode exists for operator development and smoke tests. It does not hide host files from an agent with shell access and is not appropriate for a reference-isolated official comparison. Containers must not receive a Docker socket, broad home mount, production credentials, or unrelated data.

8.2 Desk attempts

Tasks run in a fresh workspace per repetition. The default timeout comes from task.yaml unless the operator explicitly overrides it. The recorded full campaign used 2 CPUs and 6 GB per container, API-cell concurrency of five, and Codex concurrency of two. The runner allows a shutdown grace interval, so elapsed wall time can exceed the nominal task budget. Full arms contain exactly one record for every task-repetition pair.

The historical protocol is single-turn desk execution. Follow-up fields in legacy design notes do not imply that this runner implements scripted desk follow-ups. A future multi-turn desk experiment needs an explicit protocol and separate results.

8.3 Build sequences

The build runner executes the initial request followed by up to three changes. Each turn is a new one-shot invocation with the prior workspace carried forward, not a claim of preserved conversational state. Only the owner's portion of a change file is sent; its tester checklist remains outside the agent view. RESULT.json records the application URL, working admin and restricted logins, notes, start command, and port. The restricted identity must remain consistent across changes.

Self-hosted builds use a dedicated port and a local data store. Linux host networking permits the grader to reach that port but does not establish reachability from another network. A platform-backed build needs an authorized test organization. A killed container can stop a hosted process; restarting it locally supports testing but does not erase a failure of the originally handed-over public URL. Record those observations separately.

9. Grading and reporting

9.1 Desk outcome

The recorded artifact-pass field is true when all required checks pass. Checks include file existence and structure, exact identifier sets and row counts, keyed values, workbook formulas and values, text requirements, and task-specific rules. Optional checks remain diagnostic. Required-check conjunction is the headline task-level criterion; a percentage of checks is not the same as a passed task.

Workbook grading should use native LibreOffice recalculation after cached values are stripped. Set BENCH_RECALC_DOCKER_IMAGE to the supplied recalculation image. The legacy grader also has a Python-formula/cached-value fallback when no native engine is configured; this fallback is development-only and must not be presented as equivalent official scoring. Configured native-engine failures are reported, not silently converted into passes.

Execution status is independent of artifact pass. The publication retains timeouts, nonzero exits, grader-error counts, and missing usage. A passing artifact written before abnormal termination remains a recorded artifact pass, not evidence of a clean completed run. The tables show how many such cases occur. Grader-error attempts stay in the denominator; no selective rerun or omission improves the headline.

9.2 Build outcome

The build grader performs limited delivery/probe checks and generates a tester sheet for unautomated requirements. A tester must verify role boundaries, server-side authorization, invitations, live data, rules, and persistence where the checklist requires them. Score the initial checklist, the new change items, and regressions of previously passing requirements separately. Automated reachability or a login-shaped page alone does not establish working credentials or an accepted application.

9.3 Metrics

- Attempt pass rate: passing artifacts divided by every scheduled attempt in a complete arm; this release's denominator is 561 per arm.
- All-three success: tasks passing in all three repetitions, divided by 187. This is observed repeated-task consistency, not a confidence interval or long-horizon service guarantee.
- Resource measures: median elapsed time, recorded usage, and estimated model cost, with missing observations stated. Cached input is priced separately from uncached input using the bundled price assumptions.
- Build measures: accepted baseline items, core delivery criteria, new change requirements, regressions, time and cost across the sequence. No combined desk/build score is defined.

The included price table is a historical estimate input, not verified current billing advice. Missing costs are not zero. Subscription-equivalent API estimates do not equal subscription charges. Human review, infrastructure, and external platform charges are outside these model-cost estimates.

10. Validity, safety, and release boundary

Most task authoring used model assistance. Positive/negative controls, deterministic regeneration, and planted-trap checks provide mechanical validation; independent practitioner review and a human performance baseline are not established by this release. The suite was developed by an organization that also develops one of the evaluated harnesses. Exposure to the task set and benchmark-driven development limit claims about unseen generalization.

The latest result release is a recorded snapshot of the complete arms of campaign full-1. Incomplete arms, build drafts, development comparisons, and tuning pilots are excluded rather than merged into a leaderboard. Historical attempt records do not carry immutable scorer identities, and the original image tag is not an image digest. The portable release source therefore supports new runs but does not guarantee byte-for-byte reconstruction of the historical agent environment or scores.

Task generators and custom checks are executable code. Review them before running on sensitive infrastructure. Raw attempt folders can contain configuration keys, session traces, generated app credentials, and sensitive model responses; keep them private. The released ledger is deliberately allowlisted and contains neither these folders nor generated app logins. Audit-model tools can send authorized task and run contents to an external provider; they are opt-in and separate from deterministic grading.

11. Reproduction and extension

README.md contains the complete setup and run commands. Install the pinned direct Python dependencies, build the agent and recalculation images, provision benchmark-only model access, and validate the task packs. Start with a single-task smoke run before launching all 187 tasks with three repetitions. Use a new label for every campaign; the release runner refuses to overwrite an existing desk label or attempt directory.

Release verification passed the reference-solution and untouched-workspace checks for all 187 desk tasks using native recalculation, and the build-task validator passed all 20 application packs. The strict desk validator also reported 28 cases where regenerated files differ from the supplied fixture bytes, despite repeated regeneration being deterministic within the check. The supplied fixtures are preserved as the release inputs; do not replace them implicitly with regenerated versions. These notices are documented in docs/validation.md and do not establish semantic equivalence between the two byte sets.

For new comparisons, record the repository commit, task file hashes, full container digests, model and provider route, reasoning settings, installed skill/plugin inventory, time and resource budgets, dependency resolution, and scorer digest before launch. Persist all scheduled outcomes and validate matrix completeness before reporting. If a scorer changes, rescore retained artifacts into a separately named snapshot, preserving the original evidence and declaring exactly what changed.

New harnesses implement the same workspace/prompt/output adapter contract and supply usage extraction when available. New tasks require explicit deliverables, source-supported business rules, a reference solution, meaningful negative controls, and deterministic inputs. Do not weaken a task to improve a participant's score. Do not claim universal business competence from performance on this finite suite.

12. Latest full-arm results: campaign full-1

This is the recorded desk scoring snapshot from the four complete arms of the latest full benchmark campaign. Each arm covers all 187 tasks three times (561 attempts); the release contains 2,244 attempts. It is not a Proto development comparison. The overall campaign included additional incomplete arms; they are not represented as complete results here. No fully acceptance-validated full build campaign is included.

Recorded artifact scores

System	Passes / 561	Pass rate	All 3 / 187	Median min
Codex / gpt-5.6-sol	431	76.83%	126	2.25
Proto / DeepSeek V4.1 Flash	374	66.67%	90	4.29
Proto / GLM-5.3-Flash	355	63.28%	77	4.03
Proto / Qwen 3.8 Flash	381	67.91%	105	7.03

Codex / gpt-5.6-sol has the highest recorded artifact pass rate in this release. The cells use different models; these numbers do not isolate a harness effect or establish performance beyond this workload.

Repetition and execution accounting

System	R1 / 187	R2 / 187	R3 / 187	Timeouts	Nonzero exits
codex-sol	138	145	148	0	0
proto-deepseek	125	126	123	56	78
proto-glm	122	116	117	34	45
proto-qwen	129	126	126	112	117

System	Passing, abnormal exit	Grader-error attempts	Missing usage
codex-sol	0	3	0
proto-deepseek	10	5	21
proto-glm	5	2	11
proto-qwen	32	3	5

Timeout and nonzero-exit columns can overlap. Artifact pass is the stored grader verdict, not normal process completion. All attempts remain in the denominator. Grader-error counts and missing usage describe unresolved evidence; they are not silently repaired or removed.

Estimated model cost

System	Observations / 561	Mean USD, observed	Sum USD, observed
codex-sol	561	0.3906	219.1257
proto-deepseek	540	0.0617	33.3271
proto-glm	550	0.0461	25.3543
proto-qwen	556	0.0472	26.2309

Costs are recorded estimates under the historical price assumptions, not invoices or current-price claims. Means exclude missing cost observations; sums are observed sums, not complete billed totals when observations are missing. The price table is in `bench/prices.json`.

Category counts

Category	Codex sol	Proto DeepSeek	Proto GLM	Proto Qwen
bookkeeping	91 / 108	70 / 108	68 / 108	72 / 108
drafting	28 / 57	21 / 57	27 / 57	20 / 57
extraction	66 / 78	56 / 78	56 / 78	69 / 78
reformatting	78 / 78	68 / 78	66 / 78	78 / 78
reports	45 / 90	47 / 90	35 / 90	34 / 90
spreadsheet	98 / 123	94 / 123	90 / 123	94 / 123
tooling	25 / 27	18 / 27	13 / 27	14 / 27

Provenance and interpretation

The launcher names image `business-bench:v38` and runtime revision `53a303309`; it sets high reasoning effort, first-party provider routes for the three API models, native workbook recalculation, and three repetitions. These are recorded launch settings, not independently recovered per-request configurations. Model usage is retained in the ledger; the Codex usage extractor may infer the configured model name when its event stream omits it.

Original `result.json` files lack per-attempt scorer hashes. Their source file hashes are retained, but a uniform immutable historical grader cannot be established from these records. The published snapshot preserves the recorded verdicts without applying current release code retrospectively. Treat the table as descriptive campaign evidence, not a scorer-controlled causal experiment or independently certified leaderboard.

`attempts.jsonl` contains every included result with allowlisted check verdicts, execution fields, usage, costs, source hashes, and source modification timestamps. File modification times are not asserted to be run start times. `summary.json` is reproducible from that ledger. `provenance.json` defines inclusion and known gaps. Raw workspaces, homes, trace logs, and secret-bearing reviewer sheets are deliberately absent.